MEMORIA DESCRIPTIVA

ANEXO TÉCNICO

SISTEMA DE CONTROL DE ASISTENCIAS Y CALIFICACIÓN DE EXÁMENES PARA CAPACITACIONES

Índice general

1. Aspectos Generales **2**

2. Laravel, Vue y Python OMR: Arquitectura del Sistema **3**
2.1. Descripción 3
2.2. Diagrama de Flujo de Aplicación 3
2.3. Objetivos y Diseño de la Arquitectura 4

3. Infraestructura **6**

4. Modelo de Base de Datos **7**
4.1. Script SQL de la Base de Datos 9

5. Módulos del Sistema **12**
5.1. Módulo de Autenticación y Autorización 12
5.2. Módulo de Gestión de Personas 12
5.3. Módulo de Gestión de Eventos 12
5.4. Módulo de Inscripciones e Importación Masiva 12
5.5. Módulo de Toma de Asistencia 12
5.6. Módulo de Calificación OMR (Python) 13
5.7. Módulo de Reportes 13

6. Requerimientos **14**
6.1. Requerimientos del Servidor 14
6.2. Requerimientos de la Estación OMR 14
6.3. Requerimientos del Cliente Operador 15

Capítulo 1

Aspectos Generales

El **Sistema de Control de Asistencias y Calificación de Exámenes para Capacitaciones** es una solución web orientada a la gestión
integral de los eventos de capacitación, convocatorias y cursos que gestiona la organización
El sistema unifica el registro de personas, la creación de eventos, la
inscripción de asistentes, la toma de asistencia por documento de identidad y la calificación automática de exámenes presentados en hojas ópticas (OMR).

El objetivo principal del sistema es centralizar y automatizar las tareas operativas que rodean a una capacitación: padrón de inscritos, control de entrada y salida con validaciones inteligentes, evaluación objetiva mediante reconocimiento óptico de marcas y emisión de reportes oficiales en Excel y PDF. De esta manera se reducen los tiempos administrativos, se eliminan errores manuales en la tabulación de notas y se mantiene un acervo digital trazable de cada evento.

El sistema permite:

- Administrar usuarios del sistema con roles diferenciados (administrador, asignado y operador).
- Mantener un padrón único de personas identificadas por DNI.
- Crear eventos clasificados por tipo (Convocatorias, Cursos, Otros) con fecha de inicio y de finalización.
- Inscribir personas a un evento de forma individual o mediante importación masiva desde Excel.
- Registrar entradas y salidas por DNI con validación anti-duplicado de 30 segundos y control de orden lógico (no se admite una entrada sin la salida previa).
- Calcular automáticamente el estado del registro como *A TIEMPO* o *TARDANZA* en función de la hora del evento.
- Calificar exámenes en formato PDF mediante un módulo Python que detecta las burbujas marcadas, compara con el gabarito y calcula la nota.
- Generar y descargar reportes individuales en PDF (evidencia y reporte completo) y reportes consolidados en Excel.
- Soportar hasta tres operadores simultáneos con autorefresco cada cinco segundos.

Capítulo 2

Laravel, Vue y Python OMR: Arquitectura del Sistema

2.1 Descripción

El sistema se organiza en tres componentes funcionales que cooperan a través de una API REST. El **backend** está construido con Laravel 10 y expone los endpoints autenticados con Laravel Sanctum, además de un grupo público `/api/omr/*` dedicado a la integración con el calificador. El **frontend** es una Single Page Application (SPA) escrita en Vue 3 con Vuetify 3, Vue Router y el store Pinia; consume la API mediante Axios e interpreta el rol del usuario para mostrar dinámicamente las opciones del menú. El **módulo OMR** es una aplicación de escritorio en Python con interfaz CustomTkinter que procesa hojas ópticas, calcula notas y sincroniza los resultados con Laravel.

La separación responde a tres requerimientos de negocio: (1) el área administrativa necesita operar el sistema desde cualquier equipo autorizado mediante el navegador; (2) la toma de asistencia requiere baja latencia y soporte multi-operador; y (3) la calificación de exámenes debe ejecutarse en una máquina específica con escáner conectado, sin que esto interrumpa al resto de operadores.

2.2 Diagrama de Flujo de Aplicación

El flujo general de información se organiza de la siguiente manera:

1. El administrador inicia sesión en el SPA con su *nickname* y contraseña; Laravel Sanctum emite un token Bearer.
2. Se da de alta a las personas en el padrón maestro, ya sea de forma manual o por importación de Excel.
3. Se crea un evento dentro de uno de los tipos catalogados (Convocatorias, Cursos, Otros) y se establece su fecha de realización.
4. Se inscribe a las personas al evento, individualmente o en bloque mediante una hoja de Excel con DNI y datos personales.
5. El día del evento, los operadores escanean el DNI o lo digitan; el backend valida la inscripción, ejecuta la regla anti-duplicado y registra la entrada o salida con su estado correspondiente.
6. El SPA refresca la tabla de asistentes cada cinco segundos para reflejar lo que registran otros operadores.
7. Si el evento incluye examen, las hojas ópticas se digitalizan en PDF y se cargan en el módulo Python OMR.

8. El OMR detecta los seis cuadrados negros de referencia, rectifica la hoja, identifica las burbujas marcadas, calcula la nota según el gabarito y produce dos PDFs por examen (evidencia y reporte completo).

9. El OMR consulta los inscritos del evento mediante `/api/omr/eventos/{id}/inscritos`, crea la evaluación, guarda el resultado y persiste cada respuesta detectada con su porcentaje de confianza.

10. Desde el SPA se pueden listar las evaluaciones, descargar los PDFs y exportar a Excel los reportes consolidados de asistencias y de calificaciones.

2.3 Objetivos y Diseño de la Arquitectura

El diseño del sistema busca separar las responsabilidades principales para facilitar el mantenimiento, el despliegue y la operación simultánea por múltiples usuarios:

- **Backend API REST:** concentra las rutas HTTP, la autenticación, la autorización por rol y la lógica de negocio.
- **Frontend SPA:** brinda la experiencia de usuario y la operación administrativa desde cualquier equipo autorizado.
- **Módulo OMR de escritorio:** ejecuta la calificación local de los exámenes y sincroniza los resultados.
- **Base de datos relacional:** mantiene el estado consistente de personas, eventos, inscripciones, registros y evaluaciones.
- **Importación y exportación Excel:** canal estándar de intercambio con áreas administrativas externas al sistema.

Componente	Responsabilidad	Evidencia
Backend API REST	Rutas autenticadas para usuarios, personas, eventos, inscripciones, registros y evaluaciones, además del grupo público OMR.	`backend/routes/api.php`, `backend/app/Http/Controllers/`
Autenticación y autorización	Login con Sanctum, hash de contraseña, emisión de tokens y validación por rol con middleware.	`backend/app/Http/Controllers/AuthController.php`, `backend/app/Http/Middleware/RolMiddleware.php`
Acceso a datos	ORM Eloquent y migraciones MySQL para todas las entidades del dominio.	`backend/app/Models/`, `backend/database/migrations/`

Componente	Responsabilidad	Evidencia
Importación / exportación	Importadores de personas y eventos por Excel y exportadores profesionales de asistencias y evaluaciones.	`backend/app/Imports/`, `backend/app/Exports/`
Frontend SPA	Vistas Vue 3 + Vuetify, router con guards de autenticación, store Pinia y cliente Axios con interceptor de token.	`frontend/src/views/`, `frontend/src/router/index.js`, `frontend/src/axios.js`
Módulo OMR	Detección de burbujas con OpenCV, conversión PDF → imagen con Poppler, generación de reportes PDF con ReportLab.	`completo.py`
Cliente HTTP del OMR	Comunicación del módulo Python con Laravel para crear evaluaciones y guardar resultados y respuestas.	`api_client.py`, `config.py`

Capítulo 3

Infraestructura

La infraestructura del sistema está orientada a un despliegue interno dentro de la red El backend Laravel se ejecuta sobre el stack XAMPP 8.2 (Apache + PHP 8.2 + MySQL/MariaDB) en una máquina servidor; el frontend Vue se sirve mediante **Vite** en modo desarrollo o se compila para producción y se publica en el mismo Apache. El módulo OMR se ejecuta de forma local en una máquina específica equipada con escáner; gracias a **PyInstaller** puede distribuirse como un único ejecutable `.exe` con todas sus dependencias y los binarios de Poppler embebidos, lo que permite usarlo en máquinas que no tengan Python instalado.

El sistema soporta operación simultánea de hasta tres operadores en la toma de asistencia, con autorefresco cada cinco segundos del listado de asistentes. La conexión entre las máquinas cliente y el servidor se realiza mediante red local TCP/IP; en `frontend/src/axios.js` se configura el `baseURL` apuntando a la IP del servidor (por ejemplo http://192.168.1.100/capacitaciones_app/backend/api).

Elemento	Descripción
Servidor de aplicación	Equipo con Windows 10/11 y XAMPP 8.2 (Apache + PHP 8.2).
Base de datos	MySQL 8 / MariaDB sobre el esquema `capacitaciones` con cotejamiento `utf8mb4_unicode_ci`.
Servidor frontend	Node.js 18+ con Vite 6 en desarrollo (puerto 5173) o publicación estática del build en Apache para producción.
Estación OMR	Equipo con Windows, Python 3.10 y dependencias de `requirements.txt`, o el ejecutable `Sistema_Calificacion_Examenes.exe` con la carpeta `poppler/` junto al binario.
Cliente operador	Cualquier máquina con un navegador moderno (Chrome, Edge, Firefox) con acceso a la red interna.
Conectividad	Red LAN interna. El servidor expone el backend en el puerto 8000 (`php artisan serve`) o sobre el virtual host de Apache.
Concurrencia	Hasta tres operadores simultáneos con refresco automático cada cinco segundos.
Almacenamiento de PDFs	Los reportes OMR se guardan en `backend/public/resultados_omr/{evento}_{fecha}_{hora}/` con subcarpetas `evidencias/` y `reporte_completo/`.

Capítulo 4

Modelo de Base de Datos

El modelo de datos se identifica a partir de las migraciones de Laravel (`backend/database/migrations/`) y de las relaciones declaradas en los modelos Eloquent (`backend/app/Models/`). El esquema se llama `capacitaciones` y agrupa tres dominios: (1) seguridad y menú, con `usuarios`, `tipo_usuarios` y `opciones`; (2) operación de eventos y asistencias, con `personas`, `tipo_eventos`, `eventos`, `inscripciones` y `registros`; y (3) evaluaciones OMR, con `evaluaciones`, `resultados` y `respuestas_omr`. Adicionalmente Laravel mantiene la tabla `sessions` para el SPA stateful con Sanctum.

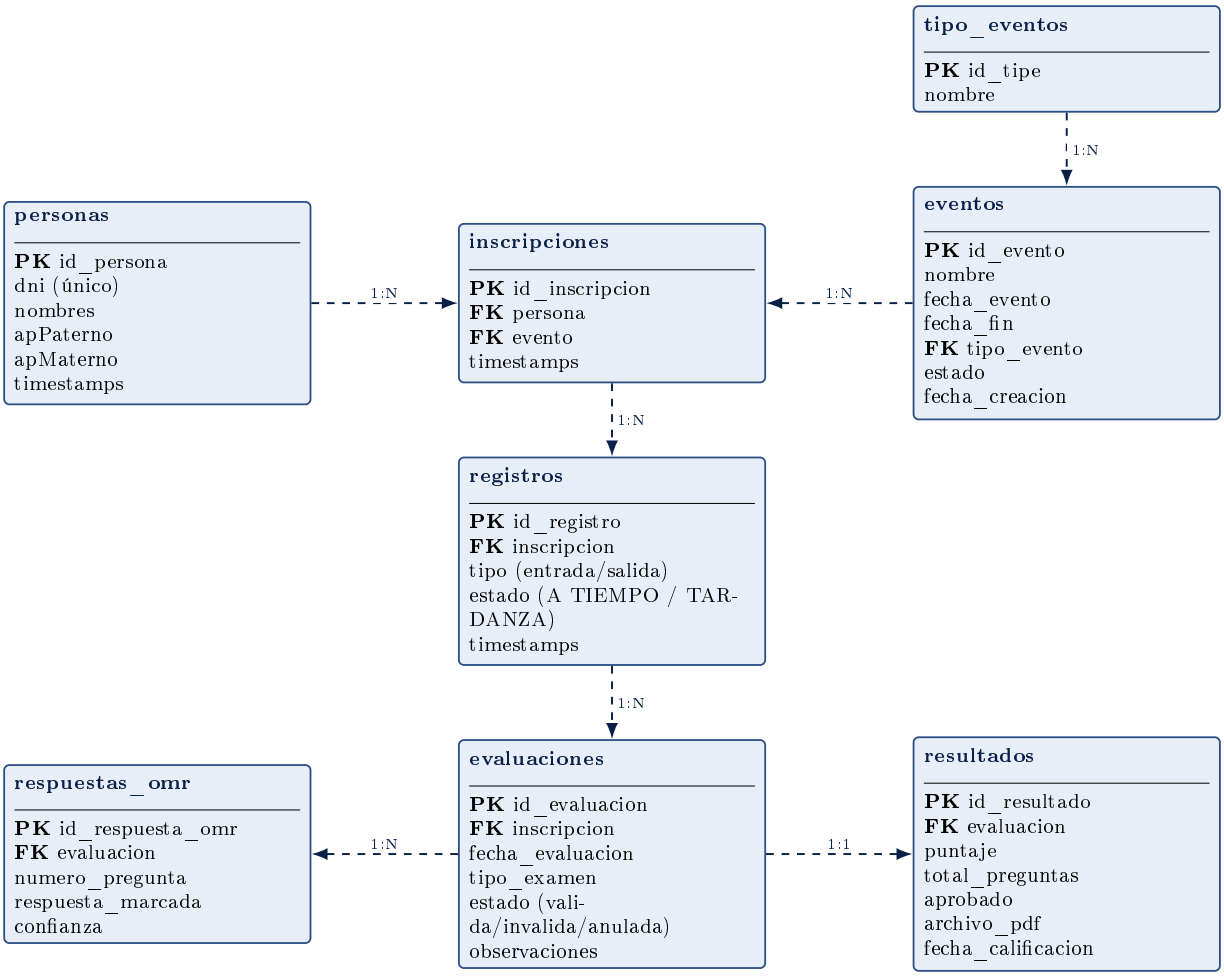


Figura 4.1: Relación lógica de las tablas principales del sistema

Tabla	Propósito	Campos principales	Relaciones
usuarios	Operadores del sistema con credenciales y rol asignado.	id_usuario, nombres_apellidos, nickname, contrasena (hash), tipo_usuario, asignado, habilitado, timestamps.	N:1 con `tipo_usuarios`; emite tokens Sanctum.
tipo_usuarios	Catálogo de roles del sistema.	id_tipu, nombre, timestamps.	1:N con `usuarios` y `opciones`.
opciones	Opciones del menú lateral mostradas según rol.	id_op, id_tipu, nombre, icono, ruta, timestamps.	N:1 con `tipo_usuarios`.
personas	Padrón maestro identificado por DNI.	id_persona, dni (único), nombres, apPaterno, apMaterno, timestamps.	1:N con `inscripciones`.
tipo_eventos	Tipos catalogados (Convocatorias, Cursos, Otros).	id_tipe, nombre.	1:N con `eventos`.
eventos	Eventos de capacitación con estado y fechas.	id_evento, nombre, fecha_evento, fecha_fin, tipo_evento, estado, fecha_creacion.	N:1 con `tipo_eventos`; 1:N con `inscripciones`.
inscripciones	Asignación de una persona a un evento.	id_inscripcion, evento, persona, timestamps.	N:1 con `eventos` y `personas`; 1:N con `registros` y `evaluaciones`.
registros	Registro de entrada o salida del asistente.	id_registro, inscripcion, tipo, estado, timestamps.	N:1 con `inscripciones`.
evaluaciones	Examen aplicado a una inscripción.	id_evaluacion, inscripcion, fecha_evaluacion, tipo_examen, estado, observaciones, timestamps.	N:1 con `inscripciones`; 1:1 con `resultados`; 1:N con `respuestas_omr`.
resultados	Nota final y archivo PDF de la evaluación.	id_resultado, evaluacion, puntaje, total_preguntas, aprobado, archivo_pdf, fecha_calificacion.	N:1 con `evaluaciones`.
respuestas_omr	Respuesta detectada por pregunta y su confianza.	id_respuesta_omr, evaluacion, numero_pregunta, respuesta_marcada, confianza.	N:1 con `evaluaciones`.

Tabla	Propósito	Campos principales	Relaciones
sessions	Sesiones de Laravel para el SPA stateful con Sanctum.	id, user_id, ip_address, user_agent, payload, last_activity.	Tabla técnica de Laravel.

4.1 Script SQL de la Base de Datos

El siguiente es el script completo de creación de la base de datos `capacitaciones` con todas las tablas, índices, restricciones de integridad referencial y datos de prueba:

[language=sql, caption=Script SQL completo de la base de datos, label=lst:sqlscript] – phpMyAdmin SQL Dump – version 5.2.1 – https://www.phpmyadmin.net/ – – Servidor: 127.0.0.1 – Tiempo de generación: 05-01-2026 a las 07:01:27 – Versión del servidor: 10.4.32-MariaDB – Versión de PHP: 8.0.30

SET SQL$_{M}ODE = "NO_{A}UTO_{V}ALUE_{O}N_{Z}ERO"; START TRANSACTION; SET time_{z}one = "+00:00";

/*!40101 SET @OLD$_{C}HARACTER_{S}ET_{C}LIENT = @@CHARACTER_{S}ET_{C}LIENT */; /*!40101 SET @OLD$_{C}HARACTER_{S}ET_{R}ESULTS = @@CHARACTER_{S}ET_{R}ESULTS */; /*!40101 SET @OLD$_{C}OLLATION_{C}ONNECTION = @@COLLATION_{C}ONNECTION */; /*!40101 SET NAMES utf8mb4 */;

– – Base de datos: ‘capacitaciones‘ –

– Estructura de tabla para la tabla ‘usuarios‘ CREATE TABLE ‘usuarios‘ (‘id$_{u}$suario‘bigint(20) UNSIGNED NOT NULL, ‘nombres$_{a}$pellidos‘varchar(255) NOT NULL, ‘nicknan
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘tipo$_{u}$suarios‘ CREATE TABLE ‘tipo$_{u}$suarios‘ (‘id$_{t}$ipu‘bigint(20)
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘opciones‘ CREATE TABLE ‘opciones‘ (‘id$_{o}$p‘bigint(20) UNSIGNED NOT NULL, ‘id$_{t}$ipu‘bigint(20) UNSIGNED NOT NULL, ‘nombre‘varch
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘personas‘ CREATE TABLE ‘personas‘ (‘id$_{p}$ersona‘bigint(20) UNSIGNED NOT NULL, ‘dni‘varchar(8) NOT NULL, ‘nombres‘varchar(100) N
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘tipo$_{e}$ventos‘ CREATE TABLE ‘tipo$_{e}$ventos‘ (‘id$_{t}$ipe‘bigint(20) UN
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘eventos‘ CREATE TABLE ‘eventos‘ (‘id$_{e}$vento‘bigint(20) UNSIGNED NOT NULL, ‘nombre‘varchar(255) NOT NULL, ‘fecha$_{e}$vento‘dateti
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘inscripciones‘ CREATE TABLE ‘inscripciones‘ (‘id$_{i}$nscripcion‘bigint(20) UNSIGNED NOT NULL, ‘evento‘bigint(20) UNSIGNED NOT NULL, ‘pers
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘registros‘ CREATE TABLE ‘registros‘ (‘id$_{r}$egistro‘bigint(20) UNSIGNED NOT NULL, ‘inscripcion‘bigint(20) UNSIGNED NOT NULL, ‘tip
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘evaluaciones‘ CREATE TABLE ‘evaluaciones‘ (‘id$_{e}$valuacion‘bigint(20) UNSIGNED NOT NULL, ‘inscripcion‘bigint(20) UNSIGNED NOT NULL, ‘
InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4$_{u}$nicode$_{c}$i;

– Estructura de tabla para la tabla ‘resultados‘ CREATE TABLE ‘resultados‘ (‘id$_{r}$esultado‘bigint(20) UNSIGNED NOT NULL, ‘evaluacion‘bigint(20) UNSIGNED NOT NULL, ‘pu

$InnoDBDEFAULTCHARSET = utf8mb4COLLATE = utf8mb4_unicode_ci$;

– Estructura de tabla para la tabla ‘respuestas$_o mr$‘$CREATETABLE$‘$respuestas_o mr$‘(‘$id_r espuesta_o$
$InnoDBDEFAULTCHARSET = utf8mb4COLLATE = utf8mb4_unicode_ci$;

– Claves primarias e índices ALTER TABLE ‘usuarios‘ ADD PRIMARY KEY (‘id$_u suario$‘), $ADDUNIQUEKEY$‘$usuarios_n ickname_u nique$‘(‘$nickname$‘), $ADDKEY$‘$usuarios_t ipo_u$

ALTER TABLE ‘tipo$_u suarios$‘$ADDPRIMARYKEY$(‘$id_t ipu$‘);

ALTER TABLE ‘opciones‘ ADD PRIMARY KEY (‘id$_o p$‘), $ADDKEY$‘$opciones_i d_t ipu_f oreign$‘(‘$id_t ipu$‘);

ALTER TABLE ‘personas‘ ADD PRIMARY KEY (‘id$_p ersona$‘), $ADDUNIQUEKEY$‘$personas_d ni_u nique$‘(‘dni‘);

ALTER TABLE ‘tipo$_e ventos$‘$ADDPRIMARYKEY$(‘$id_t ipe$‘);

ALTER TABLE ‘eventos‘ ADD PRIMARY KEY (‘id$_e vento$‘), $ADDKEY$‘$eventos_t ipo_e vento_f oreign$‘(‘$tipo_e vento$‘);

ALTER TABLE ‘inscripciones‘ ADD PRIMARY KEY (‘id$_i nscripcion$‘), $ADDKEY$‘$inscripciones_e vento_f oreign$‘(‘$evento$‘), $ADDKEY$‘$inscripciones_p ersona_f$

ALTER TABLE ‘registros‘ ADD PRIMARY KEY (‘id$_r egistro$‘), $ADDKEY$‘$registros_i nscripcion_f oreign$‘(‘$inscripcion$‘);

ALTER TABLE ‘evaluaciones‘ ADD PRIMARY KEY (‘id$_e valuacion$‘), $ADDKEY$‘$evaluaciones_i nscripcion_f oreign$‘(‘$inscripcion$‘);

ALTER TABLE ‘resultados‘ ADD PRIMARY KEY (‘id$_r esultado$‘), $ADDKEY$‘$resultados_e valuacion_f oreign$‘(‘$evaluacion$‘);

ALTER TABLE ‘respuestas$_o mr$‘$ADDPRIMARYKEY$(‘$id_r espuesta_o mr$‘), $ADDUNIQUEKEY$‘r

– AUTO$_I NCREMENTALTERTABLE$‘$usuarios$‘$MODIFY$‘$id_u suario$‘$bigint$(20)$UNSIGNEDN$
6;

ALTER TABLE ‘tipo$_u suarios$‘$MODIFY$‘$id_t ipu$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCR$
4;

ALTER TABLE ‘opciones‘ MODIFY ‘id$_o p$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCREME$
7;

ALTER TABLE ‘personas‘ MODIFY ‘id$_p ersona$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCR$
26;

ALTER TABLE ‘tipo$_e ventos$‘$MODIFY$‘$id_t ipe$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCRE$
4;

ALTER TABLE ‘eventos‘ MODIFY ‘id$_e vento$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCREM$
9;

ALTER TABLE ‘inscripciones‘ MODIFY ‘id$_i nscripcion$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO$
51;

ALTER TABLE ‘registros‘ MODIFY ‘id$_r egistro$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NCR$
2;

ALTER TABLE ‘evaluaciones‘ MODIFY ‘id$_e valuacion$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I$
23;

ALTER TABLE ‘resultados‘ MODIFY ‘id$_r esultado$‘$bigint$(20)$UNSIGNEDNOTNULLAUTO_I NC$
23;

ALTER TABLE ‘respuestas$_o mr$‘$MODIFY$‘$id_r espuesta_o mr$‘$bigint$(20)$UNSIGNEDNOTNULLA$
661;

– Restricciones de integridad referencial ALTER TABLE ‘usuarios‘ ADD CONSTRAINT ‘usuarios$_t ipo_u suario_f oreign$‘$FOREIGNKEY$(‘$tipo_u suario$‘)$REFERENCES$‘$tipo_u suarios$‘

ALTER TABLE ‘opciones‘ ADD CONSTRAINT ‘opciones$_i d_t ipu_f oreign$‘$FOREIGNKEY$(‘$id_t ipu$‘)

ALTER TABLE ‘eventos‘ ADD CONSTRAINT ‘eventos$_t ipo_e vento_f oreign$‘$FOREIGNKEY$(‘$tipo_e v$

ALTER TABLE ‘inscripciones‘ ADD CONSTRAINT ‘inscripciones$_evento_foreign‘FOREIGNKEY(‘evento‘)REFERENCES‘eventos‘(‘id_evento‘)ONDEL

ALTER TABLE ‘registros‘ ADD CONSTRAINT ‘registros$_inscripcion_foreign‘FOREIGNKEY(‘in

ALTER TABLE ‘evaluaciones‘ ADD CONSTRAINT ‘evaluaciones$_inscripcion_foreign‘FOREIGNKEY(‘inscripcion‘)REFERENCES‘inscripciones‘(‘id_i

ALTER TABLE ‘resultados‘ ADD CONSTRAINT ‘resultados$_evaluacion_foreign‘FOREIGNKEY(‘evaluacion‘)REFERENCES‘evaluaciones‘(‘id_evalu

ALTER TABLE ‘respuestas$_omr‘ADDCONSTRAINT‘respuestas_omr_evaluacion_foreign‘FOREIG

COMMIT;

Capítulo 5

Módulos del Sistema

5.1 Módulo de Autenticación y Autorización

Implementa el inicio de sesión mediante *nickname* y contraseña con `Hash::check` de Laravel, emite un token Bearer con Laravel Sanctum y devuelve al frontend la lista de `opciones` de menú filtradas por el rol del usuario. La autorización se aplica en todas las rutas protegidas mediante el middleware `RolMiddleware` que valida el campo `tipo_usuario` contra los roles permitidos en cada grupo de rutas.

5.2 Módulo de Gestión de Personas

Mantiene un padrón único identificado por DNI. Provee operaciones CRUD con validación de duplicados, búsqueda y un importador masivo desde Excel basado en Maatwebsite/Excel. El módulo desacopla la identidad de la persona de su participación en eventos, lo que permite reutilizar el padrón a lo largo del tiempo.

5.3 Módulo de Gestión de Eventos

Permite crear, editar, activar y finalizar eventos clasificados por tipo. Cada evento conserva su `fecha_evento` (apertura) y, cuando es marcado como finalizado, su `fecha_fin`. El módulo expone un importador combinado que recibe en una sola hoja Excel el evento y todas las personas a inscribir, generando los registros de `personas` e `inscripciones` en una transacción.

5.4 Módulo de Inscripciones e Importación Masiva

Asigna personas a un evento, ya sea individualmente o por importación de Excel. El importador valida los DNI, descarta duplicados, crea las personas faltantes en el padrón y genera las inscripciones correspondientes, devolviendo un resumen con totales procesados y errores.

5.5 Módulo de Toma de Asistencia

Registra entrada o salida por DNI con un conjunto estricto de reglas de negocio: (1) la persona debe estar inscrita al evento; (2) no se permite registrar dos eventos del mismo tipo en menos de 30 segundos; (3) no se admite una entrada sin la salida previa correspondiente; y (4) el estado se calcula automáticamente como *A TIEMPO* o *TARDANZA* en función de `fecha_evento`. La vista `AsistentesView.vue` se autorefresca cada cinco segundos para reflejar el trabajo de operadores concurrentes.

5.6 Módulo de Calificación OMR (Python)

Aplicación de escritorio en CustomTkinter que ejecuta el siguiente pipeline por cada hoja:

1. Carga del gabarito en formato `pregunta:respuesta:puntaje` desde un archivo de texto.
2. Conversión del PDF a imagen con `pdf2image` apoyado en los binarios de Poppler embebidos.
3. Detección de los seis cuadrados negros de referencia y rectificación de la hoja con homografía mediante `cv2.warpPerspective`.
4. Segmentación por pregunta, umbralización Otsu y selección de la burbuja con mayor área oscura por encima del umbral de confianza.
5. Comparación con el gabarito y cálculo del puntaje final.
6. Generación de dos PDFs por examen con ReportLab: una evidencia con las marcas circuladas y un reporte completo con cabecera, tabla de respuestas y firma.
7. Sincronización con Laravel mediante `api_client.py`: crea la evaluación, guarda el resultado y persiste cada respuesta detectada con su porcentaje de confianza.

5.7 Módulo de Reportes

Provee dos exportadores Excel construidos con PhpSpreadsheet y Maatwebsite/Excel: (1) el reporte de asistencias incluye datos de los asistentes, tipos de registro, estados, ancho automático de columnas y bloque de estadísticas; (2) el reporte de evaluaciones incluye puntajes, totales, condición de aprobación y fecha de calificación. Adicionalmente, el endpoint `/api/evaluaciones/pdf/{id_evento}/{dni}/{tipo}` permite descargar el PDF individual de cada examen calificado, ya sea la evidencia o el reporte completo.

Capítulo 6

Requerimientos

Por tratarse de un sistema con backend, base de datos, frontend SPA y módulo de escritorio para calificación, se consideran requerimientos diferenciados para el servidor, la estación de calificación OMR y los equipos cliente que operan la SPA.

6.1 Requerimientos del Servidor

Elemento	Requerimiento
Sistema operativo	Windows 10 / 11 (entorno principal); compatible también con Linux.
Stack web	XAMPP 8.2 con Apache, PHP 8.2 y MySQL/MariaDB.
Lenguaje backend	PHP 8.1 o superior con extensiones requeridas por Laravel 10 (mbstring, openssl, pdo, mysql, fileinfo, tokenizer, xml, ctype, json).
Base de datos	MySQL 8 o MariaDB con esquema `capacitaciones` y cotejamiento `utf8mb4_unicode_ci`.
Gestor de dependencias PHP	Composer 2.x.
Stack frontend	Node.js 18+ y npm para compilar el SPA Vue con Vite.
Memoria RAM	Mínimo 4 GB; se recomienda 8 GB para servir el SPA en desarrollo y la base de datos simultáneamente.
Almacenamiento	Espacio para la base de datos, el código fuente y los PDFs OMR generados en `backend/public/resultados_omr/`.
Conectividad	Red local con IP fija o nombre DNS estable accesible desde las máquinas cliente y la estación OMR.
Puertos	Apache (80 o 8000) para el backend; 5173 si se usa Vite en desarrollo.

6.2 Requerimientos de la Estación OMR

Elemento	Requerimiento
Sistema operativo	Windows 10 / 11 de 64 bits.

Elemento	Requerimiento
Lenguaje	Python 3.10 con las dependencias de `requirements.txt` (OpenCV, NumPy, Pillow, customtkinter, pdf2image, ReportLab, requests).
Distribución alternativa	Ejecutable empaquetado con PyInstaller (`Sistema_Calificacion_Examenes.exe`) acompañado de la carpeta `poppler/` y de los archivos `api_client.py` y `config.py`.
Hardware periférico	Escáner conectado por USB o impresora multifuncional con función de escaneo a PDF.
Memoria RAM	Mínimo 4 GB para procesar lotes de hojas escaneadas.
Conectividad	Acceso al backend Laravel (`API_BASE_URL` configurado en `config.py`).
Almacenamiento	Espacio para los PDFs originales, las imágenes intermedias y los reportes OMR generados.

6.3 Requerimientos del Cliente Operador

Elemento	Requerimiento
Sistema operativo	Cualquier sistema operativo con un navegador moderno.
Navegador	Google Chrome, Microsoft Edge o Mozilla Firefox en su versión actual.
Conexión	Acceso a la red local autorizada donde se aloja el servidor.
Resolución	1366×768 o superior; el SPA se adapta mediante el sistema de *breakpoints* de Vuetify.
Periféricos	Lector de DNI tipo escáner de código de barras (opcional, mejora la velocidad de toma de asistencia).
Concurrencia	Hasta tres operadores simultáneos por evento sin pérdida de información, gracias al refresco automático de cinco segundos.